



Ansible: Galaxy, Lint y GH Actions

Sesión #24

Presentado por:
Wilmar Rengifo

GitHub Actions y Ansible

¿Por qué combinarlos?:

- GitHub Actions orquesta la automatización al detectar cambios en el código.
- Ansible ejecuta la configuración directamente en los servidores.
- Juntos permiten un flujo DevOps completo y reproducible.

¿Qué es Ansible Galaxy?

Ansible Galaxy es una plataforma para compartir roles y colecciones de Ansible, que facilita la reutilización de código y la colaboración en la comunidad de Ansible. Es parte del ecosistema de Ansible y ofrece recursos que puedes utilizar en tus playbooks para automatizar tareas de manera más eficiente.

¿Qué es Ansible Galaxy?

Roles: Un rol en Ansible Galaxy es una forma de estructurar y organizar tareas relacionadas con una funcionalidad específica, como la instalación de un servidor web, la configuración de una base de datos, o la gestión de usuarios. Los roles incluyen tareas, variables, archivos, plantillas y handlers que puedes usar en tus proyectos.

Colecciones: Las colecciones son un conjunto más amplio que incluye roles, módulos, plugins y playbooks. Son útiles para agrupar funcionalidades que pertenecen a un dominio específico o a un proveedor, como la gestión de infraestructura en AWS, Azure, o la automatización de redes.

Usar roles o colecciones:

`ansible-galaxy install <autor>.rolename`

`ansible-galaxy collection install <namespace>.collection_name`

Uso de ansible-galaxy:

`ansible-galaxy init mi_rol`

`ansible-galaxy list`

`ansible-galaxy remove <autor>.rolename`

`ansible-galaxy search <role>`

`ansible-galaxy info <autor>.rolename`

Instalar un rol de Ansible Galaxy

```
ansible-galaxy install -p roles geerlingguy.nginx
```

```
---  
- name: Configurar servidor web  
  hosts: web  
  become: yes  
  
  roles:  
    - geerlingguy.apache
```

Uso de ansible-lint

pip3 install ansible-lint

ansible-lint site.yml -x fqcn -x idiom

Rule Violation Summary				
count	tag	profile	rule	associated tags
1	yaml[empty-lines]	basic	formatting, yaml	
1	yaml[line-length]	basic	formatting, yaml	
5	yaml[new-line-at-end-of-file]	basic	formatting, yaml	
1	yaml[octal-values]	basic	formatting, yaml	
9	yaml[truthy]	basic	formatting, yaml	
1	risky-file-permissions	safety	unpredictability	
2	ignore-errors	shared	unpredictability	
4	no-changed-when	shared	command-shell, idempotency	

Conexión usando OIDC

¿Qué es OIDC? GitHub emite un token JWT firmado por cada ejecución. AWS lo verifica y entrega credenciales temporales vía STS. No se almacena ninguna clave.

Conexión usando OIDC

- Sin secrets estáticos que rotar o filtrar
- Credenciales expiran automáticamente (15–60 min)
- Acceso restringido por repo, rama y workflow
- Auditable y alineado con el principio de mínimo privilegio

Conexión usando OIDC

- En AWS IAM → Identity providers → Add provider → OpenID Connect → URL: `https://token.actions.githubusercontent.com` → Audience: `sts.amazonaws.com`
- En IAM → Roles → Create role → Web identity → Selecciona el provider → Audience: `sts.amazonaws.com`
- En el rol → Trust policy → agrega condición
`"token.actions.githubusercontent.com:sub":`
`"repo:TU_ORG/TU_REPO:ref:refs/heads/main"`
- Adjunta las políticas que necesita el rol de GitHub Actions (EC2, SSM, S3, Secrets Manager, etc.)

Conexión usando OIDC

- Copia el ARN del rol y úsalo en el workflow con role-to-assume
- Agrega los permisos id-token: write y contents: read al workflow
- En IAM → Roles → Create role → AWS service → EC2 → adjunta AmazonSSMManagedInstanceCore → este es el rol para las instancias
- En EC2 → cada instancia → Actions → Security → Modify IAM role → asigna el Instance Profile del paso 7
- Verifica que el SSM Agent esté instalado y corriendo en las instancias (systemctl status amazon-ssm-agent)